

```
-- =====  
-- SQL DATA ANALYST — PRACTICAL SHORT NOTES  
-- =====
```

-- 01. SELECT

-- Data ko read/dekhne ke liye

```
SELECT * FROM employees;
```

```
SELECT name, salary, department  
FROM employees;
```

-- 02. DISTINCT

-- Duplicate values hata kar unique values

```
SELECT DISTINCT department  
FROM employees;
```

-- 03. WHERE

-- Specific rows filter karna

```
SELECT *  
FROM employees  
WHERE salary > 50000;
```

```
SELECT *
```

```
FROM employees
WHERE department = 'IT';
```

-- 04. COMPARISON OPERATORS

```
-- = equal
-- > greater
-- < smaller
-- >= greater/equal
-- <= smaller/equal
-- <> not equal
```

```
SELECT *
FROM employees
WHERE salary >= 60000;
```

-- 05. AND / OR

-- Multiple conditions

```
SELECT *
FROM employees
WHERE salary > 50000
AND department = 'IT';
```

```
SELECT *
FROM employees
WHERE department = 'IT'
OR department = 'HR';
```

-- 06. NOT

-- Condition ko reverse karna

```
SELECT *
```

```
FROM employees
```

```
WHERE NOT department = 'HR';
```

-- 07. IN

-- Multiple possible values

```
SELECT *
```

```
FROM employees
```

```
WHERE department IN ('IT', 'HR', 'Sales');
```

-- 08. BETWEEN

-- Range ke andar values

```
SELECT *
```

```
FROM employees
```

```
WHERE salary BETWEEN 40000 AND 80000;
```

-- 09. LIKE

-- Pattern/search matching

```
SELECT *  
FROM employees  
WHERE name LIKE 'A%';    -- A se start
```

```
SELECT *  
FROM employees  
WHERE name LIKE '%a';    -- a par end
```

```
SELECT *  
FROM employees  
WHERE name LIKE '%an%';  -- "an" kahin bhi
```

```
-- 10. ORDER BY  
-- Sorting
```

```
SELECT *  
FROM employees  
ORDER BY salary ASC;
```

```
SELECT *  
FROM employees  
ORDER BY salary DESC;
```

```
-- 11. LIMIT  
-- Sirf top N rows
```

```
SELECT *
```

```
FROM employees
ORDER BY salary DESC
LIMIT 10;
```

```
-- =====
```

```
-- AGGREGATE FUNCTIONS
```

```
-- =====
```

```
-- 12. COUNT
```

```
-- Rows/count
```

```
SELECT COUNT(*)
FROM employees;
```

```
-- Condition ke saath
```

```
SELECT COUNT(*)
FROM employees
WHERE department = 'IT';
```

```
-- 13. SUM
```

```
-- Total
```

```
SELECT SUM(salary)
FROM employees;
```

```
-- 14. AVG
```

-- Average

```
SELECT AVG(salary)
FROM employees;
```

-- 15. MIN / MAX

-- Lowest / Highest

```
SELECT MIN(salary)
FROM employees;
```

```
SELECT MAX(salary)
FROM employees;
```

-- =====

-- GROUPING

-- =====

-- 16. GROUP BY

-- Category-wise calculation

```
SELECT department, COUNT(*) AS employee_count
FROM employees
GROUP BY department;
```

```
SELECT department, AVG(salary) AS avg_salary
FROM employees
```

GROUP BY department;

-- 17. GROUP BY + SUM

-- Department-wise total salary

```
SELECT department,  
       SUM(salary) AS total_salary  
FROM employees  
GROUP BY department;
```

-- 18. HAVING

-- GROUP BY ke result ko filter karna

```
SELECT department,  
       AVG(salary) AS avg_salary  
FROM employees  
GROUP BY department  
HAVING AVG(salary) > 60000;
```

-- 🧠 WHERE vs HAVING

--

-- WHERE → rows filter

-- HAVING → groups filter

-- =====

-- JOINS

-- =====

-- 19. INNER JOIN

-- Dono tables mein matching records

```
SELECT e.name,  
       d.department_name  
FROM employees e  
INNER JOIN departments d  
ON e.department_id = d.department_id;
```

-- 20. LEFT JOIN

-- Left table ke saare records

```
SELECT e.name,  
       d.department_name  
FROM employees e  
LEFT JOIN departments d  
ON e.department_id = d.department_id;
```

-- 21. LEFT JOIN + NULL

-- Jinka matching record nahi mila

```
SELECT e.*  
FROM employees e  
LEFT JOIN departments d
```



```
ON e.department_id = d.department_id
WHERE d.department_id IS NULL;
```

```
-- 🧠 JOIN TRIGGER
```

```
--
```

```
-- "2 tables ka data combine karna hai"
```

```
--      ↓
```

```
--      JOIN
```

```
--
```

```
-- Matching only → INNER JOIN
```

```
-- Left ka sab → LEFT JOIN
```

```
-- =====
```

```
-- NULL
```

```
-- =====
```

```
-- 22. NULL CHECK
```

```
SELECT *
```

```
FROM employees
```

```
WHERE salary IS NULL;
```

```
SELECT *
```

```
FROM employees
```

```
WHERE salary IS NOT NULL;
```

-- 23. COALESCE

-- NULL ki jagah default value

```
SELECT name,  
       COALESCE(phone, 'Not Available') AS phone  
FROM employees;
```

-- =====

-- CASE WHEN

-- =====

-- 24. CASE

-- SQL mein IF/ELSE jaisa

```
SELECT name,  
       salary,  
       CASE  
         WHEN salary >= 80000 THEN 'High'  
         WHEN salary >= 50000 THEN 'Medium'  
         ELSE 'Low'  
       END AS salary_level  
FROM employees;
```

-- Example: Sales performance

```
SELECT employee_id,  
       sales,
```

```

CASE
    WHEN sales >= 100000 THEN 'Excellent'
    WHEN sales >= 50000 THEN 'Good'
    ELSE 'Needs Improvement'
END AS performance
FROM sales;

-- =====
-- SUBQUERY
-- =====

-- 25. SUBQUERY
-- Query ke andar query

-- Average salary se zyada salary wale employees

SELECT *
FROM employees
WHERE salary > (
    SELECT AVG(salary)
    FROM employees
);

-- =====
-- CTE
-- =====

```

-- 26. CTE

-- Complex query ko readable banana

```
WITH high_salary AS (  
    SELECT *  
    FROM employees  
    WHERE salary > 80000  
)  
SELECT *  
FROM high_salary;
```

-- Multiple steps ko separate kar sakte ho

```
WITH sales_summary AS (  
    SELECT employee_id,  
           SUM(amount) AS total_sales  
    FROM sales  
    GROUP BY employee_id  
)  
SELECT *  
FROM sales_summary  
WHERE total_sales > 100000;
```

-- 🧠 TRIGGER

-- Query bahut complex ho rahi hai

-- ↓

-- CTE / WITH

```
-- =====
```

```
-- DUPLICATES
```

```
-- =====
```

```
-- 27. Duplicate values find karna
```

```
SELECT email,  
       COUNT(*) AS total  
FROM employees  
GROUP BY email  
HAVING COUNT(*) > 1;
```

```
-- =====
```

```
-- WINDOW FUNCTIONS
```

```
-- =====
```

```
-- 28. ROW_NUMBER
```

```
-- Har row ko unique number
```

```
SELECT name,  
       salary,  
       ROW_NUMBER() OVER (  
         ORDER BY salary DESC  
       ) AS row_num  
FROM employees;
```

-- 29. RANK

-- Same salary → same rank

-- Gap aa sakta hai

```
SELECT name,  
       salary,  
       RANK() OVER (  
         ORDER BY salary DESC  
       ) AS rank_num  
FROM employees;
```

-- 30. DENSE_RANK

-- Same salary → same rank

-- Gap nahi aata

```
SELECT name,  
       salary,  
       DENSE_RANK() OVER (  
         ORDER BY salary DESC  
       ) AS rank_num  
FROM employees;
```

-- 31. DEPARTMENT-WISE RANKING

```
SELECT name,  
       department,
```

```
salary,  
RANK() OVER (  
    PARTITION BY department  
    ORDER BY salary DESC  
    ) AS rank_num  
FROM employees;
```

```
-- 🧠 PARTITION BY  
-- Data ko groups mein divide karta hai  
-- Har group ke andar calculation/ranking
```

```
-- =====  
-- TOP N / SECOND HIGHEST  
-- =====
```

```
-- 32. Highest salary
```

```
SELECT MAX(salary)  
FROM employees;
```

```
-- 33. Second highest salary
```

```
SELECT MAX(salary)  
FROM employees  
WHERE salary < (  
    SELECT MAX(salary)
```

```
FROM employees
);
```

```
-- 34. Nth highest salary
-- DENSE_RANK use karo
```

```
WITH ranked AS (
    SELECT salary,
           DENSE_RANK() OVER (
               ORDER BY salary DESC
           ) AS rnk
    FROM employees
)
SELECT salary
FROM ranked
WHERE rnk = 3;
```

```
-- =====
-- LAG / LEAD
-- =====
```

```
-- 35. LAG
-- Previous row ki value
```

```
SELECT date,
       sales,
       LAG(sales) OVER (
```



```
        ORDER BY date
    ) AS previous_sales
FROM sales;
```

```
-- 36. LEAD
-- Next row ki value
```

```
SELECT date,
       sales,
       LEAD(sales) OVER (
           ORDER BY date
       ) AS next_sales
FROM sales;
```

```
-- =====
-- RUNNING TOTAL
-- =====
```

```
-- 37. Running total
```

```
SELECT date,
       sales,
       SUM(sales) OVER (
           ORDER BY date
       ) AS running_total
FROM sales;
```

```
-- =====
```

```
-- PARTITION + AGGREGATION
```

```
-- =====
```

```
-- 38. Har department ka total salary
```

```
-- Har employee ke row mein department total dikhega
```

```
SELECT name,  
       department,  
       salary,  
       SUM(salary) OVER (  
           PARTITION BY department  
       ) AS department_total  
FROM employees;
```

```
-- =====
```

```
-- DATE QUERIES
```

```
-- =====
```

```
-- 39. Date filter
```

```
SELECT *  
FROM orders  
WHERE order_date >= '2026-01-01';
```

```
-- 40. YEAR-WISE SALES
```

```
SELECT YEAR(order_date) AS year,  
       SUM(amount) AS total_sales  
FROM orders  
GROUP BY YEAR(order_date);
```

-- 41. MONTH-WISE SALES

```
SELECT MONTH(order_date) AS month,  
       SUM(amount) AS total_sales  
FROM orders  
GROUP BY MONTH(order_date);
```

```
-- =====  
-- STRING FUNCTIONS  
-- =====
```

-- 42. UPPER / LOWER

```
SELECT UPPER(name)  
FROM employees;
```

```
SELECT LOWER(name)  
FROM employees;
```

-- 43. LENGTH

```
SELECT name,  
       LENGTH(name) AS name_length  
FROM employees;
```

```
-- 44. TRIM  
-- Extra spaces remove
```

```
SELECT TRIM(name)  
FROM employees;
```

```
-- =====  
-- CONDITIONAL AGGREGATION  
-- =====
```

```
-- 45. COUNT with condition
```

```
SELECT  
    COUNT(CASE  
        WHEN salary >= 50000 THEN 1  
        END) AS high_salary_count  
FROM employees;
```

```
-- 46. SUM with condition
```

```
SELECT
```

```
SUM(CASE
    WHEN department = 'IT' THEN salary
    ELSE 0
END) AS it_salary
FROM employees;
```

```
-- =====
-- UNION
-- =====
```

```
-- 47. UNION
-- Do queries ke results combine
-- Duplicate remove karega
```

```
SELECT name FROM employees
UNION
SELECT name FROM managers;
```

```
-- 48. UNION ALL
-- Duplicate bhi rakhega
```

```
SELECT name FROM employees
UNION ALL
SELECT name FROM managers;
```

```
-- =====
```

-- EXISTS

-- =====

-- 49. EXISTS

-- Check karta hai related record exist karta hai ya nahi

SELECT *

FROM customers c

WHERE EXISTS (

 SELECT 1

 FROM orders o

 WHERE o.customer_id = c.customer_id

);

-- =====

-- UPDATE / DELETE / INSERT

-- =====

-- 50. INSERT

INSERT INTO employees

(name, department, salary)

VALUES

('Rahul', 'IT', 60000);

-- 51. UPDATE

```
UPDATE employees
SET salary = 65000
WHERE employee_id = 101;
```

```
-- 52. DELETE
```

```
DELETE FROM employees
WHERE employee_id = 101;
```

```
-- ⚠ IMPORTANT:
```

```
-- UPDATE/DELETE mein WHERE bhoolna dangerous hai.
```

```
-- =====
```

```
-- MOST IMPORTANT QUERY PATTERNS
```

```
-- =====
```

```
-- Filter
```

```
--    → WHERE
```

```
-- Category-wise calculation
```

```
--    → GROUP BY
```

```
-- Group ko filter
```

```
--    → HAVING
```

```
-- Tables combine
```

-- → JOIN

-- Query ke andar query

-- → SUBQUERY

-- Complex query

-- → CTE

-- Ranking

-- → RANK / DENSE_RANK

-- Unique numbering

-- → ROW_NUMBER

-- Previous value

-- → LAG

-- Next value

-- → LEAD

-- Running total

-- → SUM() OVER()

-- Duplicate find

-- → GROUP BY + HAVING COUNT(*) > 1

-- Missing matching records

-- → LEFT JOIN + IS NULL

-- Nth highest

-- → DENSE_RANK()

-- Condition ke according category

-- → CASE WHEN

-- NULL replace

-- → COALESCE()

-- =====

-- 🧠 SQL QUERY THINKING ORDER

-- =====

SELECT

FROM

JOIN

WHERE

GROUP BY

HAVING

ORDER BY

LIMIT;

-- Meaning:

-- SELECT → KYA chahiye?

-- FROM → KAHA se?

-- JOIN → KAUNSI table add karni?

-- WHERE → KAUNSI rows?
-- GROUP BY → KIS GROUP mein?
-- HAVING → KAUNSE GROUP?
-- ORDER BY → KIS ORDER mein?
-- LIMIT → KITNE RESULTS?

-- =====

-- 🔥 GOLDEN SQL FORMULA

-- =====

-- Question padho

-- ↓

-- Data kahan hai? → FROM

-- ↓

-- Table combine? → JOIN

-- ↓

-- Rows filter? → WHERE

-- ↓

-- Calculation? → SUM/AVG/COUNT

-- ↓

-- Category-wise? → GROUP BY

-- ↓

-- Group filter? → HAVING

-- ↓

-- Ranking/previous? → WINDOW FUNCTION

-- ↓

-- Sorting? → ORDER BY

-- ↓

-- Top N? → LIMIT

question samajhne ke tarike

"IT department ke employees"

↓

WHERE

"department-wise average salary"

↓

GROUP BY + AVG

"average salary se zyada"

↓

SUBQUERY

"department ke andar highest salary"

↓

PARTITION BY + RANK

"second highest salary"

↓

DENSE_RANK / SUBQUERY

"previous month's sales"

↓

LAG

"running total"

↓

SUM() OVER()

"jinka order nahi hai"



LEFT JOIN + IS NULL

"duplicate emails"



GROUP BY + HAVING COUNT(*) > 1